

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
C0 4 ASSESSMENT TOOL 1

Course Code:	ITA0609	Course Title:	Machine Learning
CO Assessed:	CO 4 – Analyze (BL 3)	Assessment:	Assessment Tool 1 30 (3 Questions ×10 Mark)
Weightage:	40% of CIA	Total Marks:	
CO1 Statement:	<i>To acquire a foundational understanding of key machine learning concepts including version spaces, candidate elimination, inductive bias, and heuristic search (BL1)</i>		
Student Name:	Sameer Ahmed G	Reg. No.:	192421154
Section / Batch:	SLOT A	Date:	12-08-2026

Code of Conduct

I (Sameer Ahmed G-192421154) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

1. Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

K-Nearest Neighbour (KNN)

Concept

- KNN is a **lazy learning algorithm** that classifies a new data point based on the majority class of its nearest neighbors.
- Distance metric: **Euclidean distance** is commonly used to measure closeness.
- The parameter **K** decides how many neighbors are considered.
 - Small K → sensitive to noise.
 - Large K → smoother decision boundary.

Steps

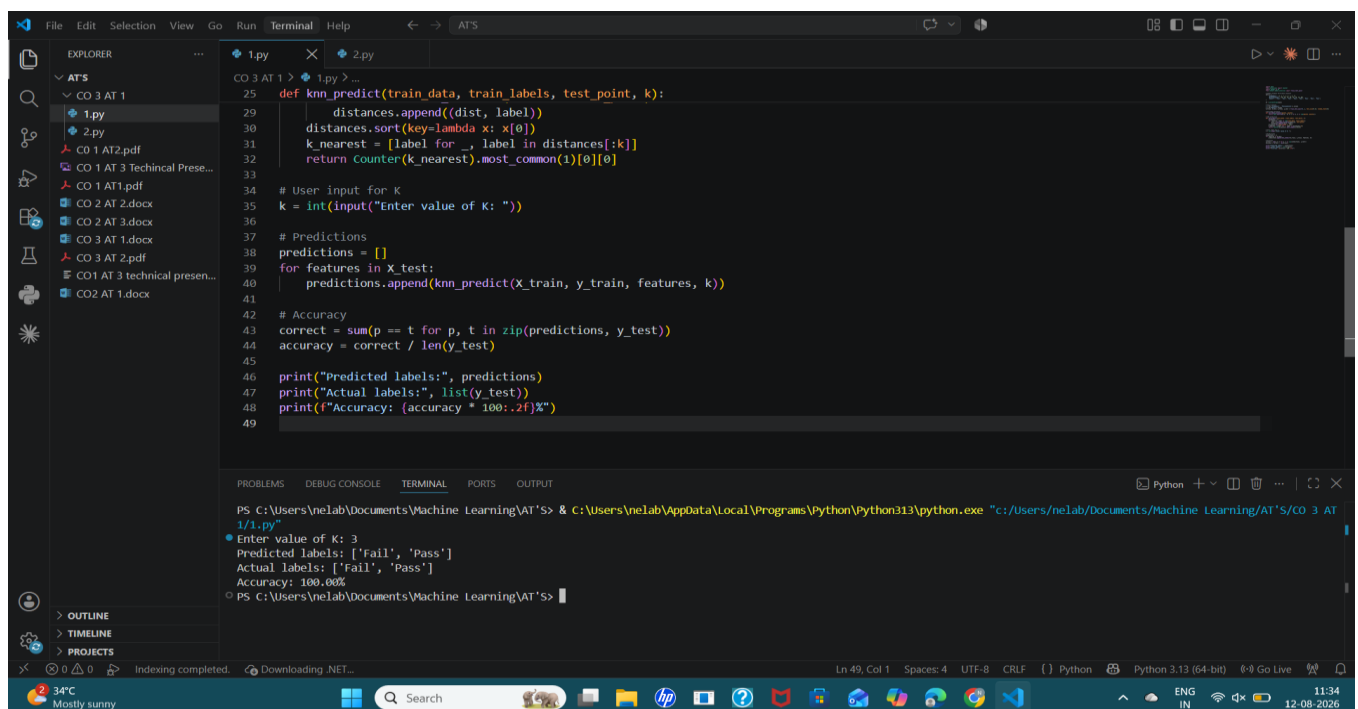
1. Store the dataset (StudyHours, PreviousScore, Result).
2. For a test student, compute Euclidean distance to all training students.
3. Sort distances and pick the **K nearest neighbors**.
4. Use majority voting to assign the class (Pass/Fail).
5. Evaluate predictions against actual labels to compute **accuracy**.

Advantages

- Simple and intuitive.
- Works well for small datasets.
- No training phase, only prediction.

Limitations

- Computationally expensive for large datasets.
- Sensitive to irrelevant features and scaling.



```
File Edit Selection View Go Run Terminal Help
EXPLORER
  ATS
    CO 3 AT 1
      1.py
      2.py
    CO 1 AT2.pdf
    CO 1 AT 3 Technical Prese...
    CO 1 AT1.pdf
    CO 2 AT 2.docx
    CO 2 AT 3.docx
    CO 3 AT 1.docx
    CO 3 AT 2.pdf
    CO 1 AT 3 technical presen...
    CO 2 AT 1.docx
  OUTLINE
  TIMELINE
  PROJECTS
  PROBLEMS DEBUG CONSOLE TERMINAL PORTS OUTPUT
  Python Python 3.13 (64-bit) Go Live
  Ln 49, Col 1 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit) Go Live 11:34 12-08-2026

25 def knn_predict(train_data, train_labels, test_point, k):
26     distances.append((dist, label))
27     distances.sort(key=lambda x: x[0])
28     k_nearest = [label for _, label in distances[:k]]
29     return Counter(k_nearest).most_common(1)[0][0]
30
31 # User input for K
32 k = int(input("Enter value of K: "))
33
34 # Predictions
35 predictions = []
36 for features in X_test:
37     predictions.append(knn_predict(x_train, y_train, features, k))
38
39 # Accuracy
40 correct = sum(p == t for p, t in zip(predictions, y_test))
41 accuracy = correct / len(y_test)
42
43 print("Predicted labels:", predictions)
44 print("Actual labels:", list(y_test))
45 print(f"Accuracy: {accuracy * 100:.2f}%")
46
47 PS C:\Users\neLab\Documents\Machine Learning\AT'S> & C:\Users\neLab\AppData\Local\Programs\Python\Python313\python.exe "c:\Users\neLab\Documents\Machine Learning\AT'S\CO 3 AT 1\1.py"
Enter value of K: 3
Predicted labels: ['fail', 'Pass']
Actual labels: ['fail', 'Pass']
Accuracy: 100.00%
```

2. Write a Python program to implement **Logistic Regression** for binary classification. Train the model using gradient descent and predict outputs for test data. Display the sigmoid function output and classification results. Evaluate the model using accuracy and discuss how learning rate affects convergence and performance.

Logistic Regression

Concept

- Logistic Regression is a **linear model for binary classification**.
- It uses the **sigmoid function** to map input values into probabilities between 0 and 1.
- Decision rule:
 - Probability $\geq 0.5 \rightarrow$ Class 1 (Pass)
 - Probability $< 0.5 \rightarrow$ Class 0 (Fail)

Steps

1. Initialize weights and bias.
2. Compute linear combination: $z = w \cdot X + b$.
3. Apply sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$.
4. Update weights using **gradient descent**:
 - $w = w - \alpha \cdot dw$
 - $b = b - \alpha \cdot db$ where α is the learning rate.
5. Repeat for multiple epochs until convergence.
6. Predict labels and evaluate accuracy.

Learning Rate Effect

- **Small learning rate (0.001)**: Slow convergence, stable training.
- **Optimal learning rate (0.1)**: Smooth convergence, good accuracy.
- **Large learning rate (1.0)**: Overshooting, oscillations, poor convergence.

Advantages

- Probabilistic interpretation (gives confidence scores).
- Efficient for linearly separable data.
- Easy to implement and interpret.

Limitations

- Assumes linear decision boundary.
- Not suitable for complex non-linear problems without feature engineering.

The image shows a Visual Studio Code editor window with a Python script for logistic regression. The script is named `2.py` and is located in the `AT'S` directory. The script imports `train_test_split` from `sklearn.model_selection` and defines a dataset `data` with features `StudyHours` and `PreviousScore`, and a target variable `Result`. The data is converted to a `DataFrame` and split into training and testing sets. A sigmoid function is defined, and a logistic regression model is trained. The output of the model is displayed in the terminal.

```
1 from sklearn.model_selection import train_test_split
2
3 # Student Performance dataset
4 data = {
5     'StudyHours': [2, 3, 4, 5, 6, 7, 8, 9],
6     'PreviousScore': [40, 50, 55, 60, 65, 70, 75, 80],
7     'Result': [0, 0, 0, 1, 1, 1, 1, 1] # 0 = Fail, 1 = Pass
8 }
9
10 df = pd.DataFrame(data)
11
12 # Features and labels
13 X = df[['StudyHours', 'PreviousScore']].values
14 y = np.array(df['Result'])
15
16 # Train-test split
17 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
18
19 # Sigmoid function
20 def sigmoid(z):
21     return 1 / (1 + np.exp(-z))
22
23 # Logistic Regression training
24 def logistic_regression(X, y, lr=0.01, epochs=1000):
25     m, n = X.shape
```

The terminal output shows the following results:

```
PS C:\Users\neLab\Documents\Machine Learning\AT'S> & C:\Users\neLab\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/neLab/Documents/Machine Learning/AT'S/CO 3 AT 1/2.py"
Sigmoid outputs: [1.48577779e-09 1.00000000e+00]
Predicted labels: [0, 1]
Actual labels: [np.int64(0), np.int64(1)]
Accuracy: 100.00%
```

Note: A smaller learning rate converges slowly but stably.
A larger learning rate may overshoot and fail to converge.

PS C:\Users\neLab\Documents\Machine Learning\AT'S>